Hazard-First Assistive Vision: A Priority-Scheduled, Schema-Verified Perception Pipeline for Visually Impaired Navigation

[[CAMERA-READY: author block]]

Abstract — Independent indoor movement confronts a visually impaired person with information that arrives silently: a descending staircase, an obstacle at walking height, a sign that is rotated or perspective-distorted. Systems that only read text or only detect objects address fragments of this problem, and most speak their findings in arrival order rather than in order of danger. We present an assistive vision system, implemented as a smartphone web client and a Python inference server, that integrates oriented scene-text reading, obstacle detection over a navigation-specific class schema, monocular distance estimation expressed in walking steps, and a priority scheduler that speaks hazards before requested objects, signs, and scene summaries. Two design decisions distinguish the system. First, *safety by construction*: the runtime identifies detector weights by their class names and refuses unrecognized vocabularies, and the dataset tooling refuses a split that declares a class it cannot populate. Second, *honest capability reporting*: the health endpoint discloses which hazard classes the deployed weights can and cannot raise. Two preliminary detectors are fully measured: a six-class model raises frame-level hazard recall from 0.252 to 0.463 over a COCO baseline at higher precision, and a seven-class model — trained after a human-verified re-tagging protocol gave the safety-critical descending-stairs class its first labelled data — reaches hazard recall 0.554 at precision 0.976. Both are withheld from deployment because person recall regresses at the deployed operating point, a negative result we report rather than average away. The pipeline itself is validated by 152 automated tests; final GPU training is in progress.

Keywords — assistive technology; visually impaired navigation; object detection; scene text recognition; hazard prioritization; monocular distance estimation; system safety verification.

I. INTRODUCTION

The World Health Organization estimates that more than two billion people live with a vision impairment, a substantial fraction of whom cannot resolve the visual cues that sighted pedestrians use continuously: door frames, staircase edges, overhead signage, and the position of people moving nearby [1]. Indoors, where satellite positioning is unavailable and tactile paving is rare, the white cane covers roughly a one-metre radius at floor level and nothing above it, and it reads no text at all.

Computer vision can, in principle, supply the missing information. Modern object detectors localize dozens of object categories in real time [9], [17], and scene-text systems detect and recognize signage under rotation and perspective distortion [2]–[5]. In practice, however, assistive deployments of these components tend to inherit two structural weaknesses. The first is fragmentation: reading applications read, detection applications detect, and neither orders its output by consequence, so the user hears about a menu board and an approaching staircase with equal urgency. The second is silent capability loss: when a detector is swapped or retrained, nothing in a typical serving stack verifies that the new weights can still raise every warning the system's logic assumes — a failure mode that is invisible precisely to the user it endangers.

This paper describes an end-to-end system built around those two observations. A smartphone browser captures frames and voice commands; a server runs detection, oriented text reading, and spatial reasoning; a priority engine composes a single spoken sentence in which danger always precedes convenience. The class schema is navigation-specific — it names staircases, doors, and dustbins that COCO-trained detectors structurally cannot report — and the runtime treats the schema as a contract to be verified, not a filename to be trusted.

Our contributions are: **(1)** a priority-scheduled perception-to-speech pipeline in which hazards interrupt, a critical class (descending stairs) preempts everything, and distant hazard-class objects are demoted so that only proximate danger interrupts the user; **(2)** step-metric monocular distance estimation that fuses two independent estimators — a class-height pinhole model and a ground-plane model with device-pitch correction — by taking the conservative minimum, reported in walking steps, with a two-minute per-device field-of-view calibration; **(3)** a safety-by-construction mechanism spanning runtime and tooling: name-based schema verification that refuses unrecognized weights, split construction that refuses unlearnable classes, and a health endpoint that reports which alerts the deployed model can actually raise; **(4)** a reproducible, leakage-safe dataset pipeline for the navigation schema, including a human-verified re-tagging protocol that gave the safety-critical descending-stairs class its first labelled data, and a measured seven-class detector trained on it. All pipeline logic is pinned by 152 automated tests that run in continuous integration.

II. RELATED WORK

Assistive vision systems. Surveys of assistive technology for blind and low-vision users trace a progression from sonar canes to camera-based scene description [6], [7]. Deployed reading assistants — exemplified by smartphone applications that narrate text and objects — demonstrate demand, but published descriptions emphasize recognition accuracy over information ordering: output is typically spoken in detection order, and obstacle awareness, when present, is not integrated with text reading in a single prioritized channel. Navigation-focused prototypes conversely emphasize obstacle avoidance and path planning [7] but rarely read signage, although signage is how buildings communicate routes to everyone else.

Scene-text detection and recognition. EAST [2] and CRAFT [3] established efficient oriented and character-

affinity text detection; DBNet [4] and MOST [5] refined arbitrary-shape detection. Recognition commonly follows the CRNN design [10]. These systems target benchmark protocols, not assistive delivery: they stop at bounding quadrilaterals and transcripts. Our pipeline consumes their output — each detected quadrilateral is perspective-rectified to horizontal before recognition — and treats reading as one prioritized voice among several, not as the product.

Object detection for navigation classes. The single-stage detection lineage from YOLO [17] to YOLOv8 [9] makes real-time obstacle detection practical on modest hardware, but the standard training vocabularies do not serve navigation. COCO [8] supplies person, furniture, and vehicle categories with massive supervision, but contains no staircase, door-as-obstacle, or dustbin class; Open Images [11] contains a single undirected *Stairs* class. Since a descending staircase is the highest-consequence indoor hazard and an ascending one is merely an obstacle, directionality is not a labelling nicety but the difference between "caution" and "stop." We are not aware of prior assistive work that both trains a directional stairs class and verifies, at runtime, that the deployed weights still carry it — the gap this paper addresses. Table I positions the work.

TABLE I — Positioning relative to representative approaches

Approach	Text	Obstacles	Prioritized speech	Capability verification
Reading assistants (e.g., [6])	yes	no	no	no
Navigation prototypes [7]	no	yes	partial	no
Text-detection literature [2]–[5]	yes	—	—	—
This work	yes (oriented)	yes (nav schema)	yes	yes

III. PROPOSED SYSTEM

The system is client–server (Fig. 1). The client is a browser application on an ordinary smartphone: it captures JPEG frames (continuous scan every ≈2.6 s in live-assist mode, or single-shot), performs speech recognition for commands, renders text-to-speech and vibration, and reads the device gyroscope and GPS. The server, a Python/FastAPI process, exposes one analysis endpoint: a frame plus an optional keyword returns detected objects, hazards, texts, symbols, spatial attributes, and a single composed speech string. Outdoor turn-by-turn navigation is delegated to a maps application by design; the contribution of this work is indoor perception, and re-implementing routing would add no information a maps client does not already speak.

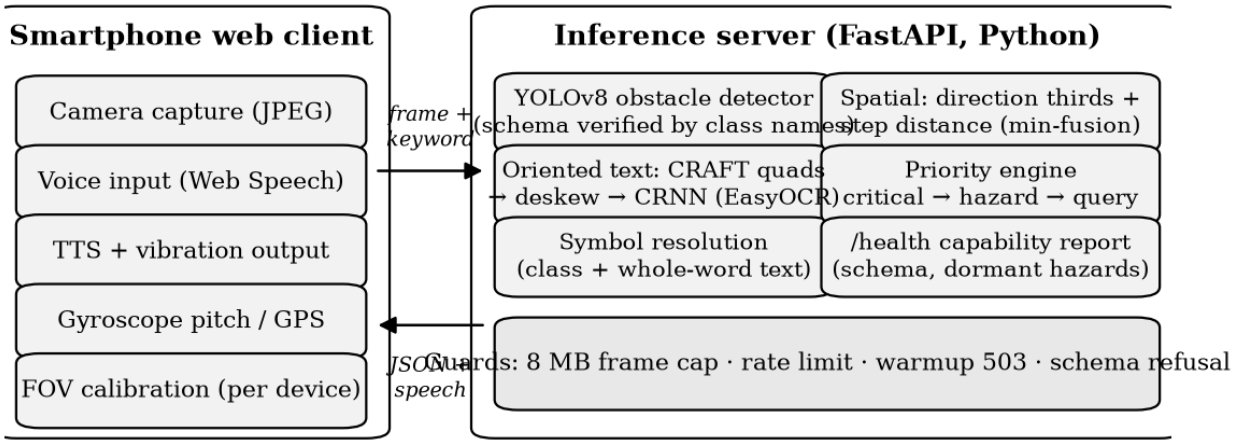

Fig. 1. Overall architecture of the proposed system.

Processing follows five stages on the server: (i) decode and validate the frame; (ii) detect obstacles with YOLOv8 over the active class schema; (iii) detect and read scene text (oriented quadrilaterals, rectified before recognition); (iv) resolve symbols and compute spatial attributes — direction and step distance — for every detection; (v) compose one prioritized sentence. The decision logic of stage (v) is deliberately simple and fully tested: danger first, then what the user asked for, then everything else.

The server defends its own availability: frames above 8 MB are refused, per-client request rate is limited, and a health endpoint reports model warmup, the active class schema, and — as discussed in §IV-D — which alert paths the loaded weights can actually serve. A failed model load reports unhealthy rather than degrading silently, so an orchestrator restarts a worker that could never answer.

IV. METHODOLOGY

A. Obstacle detection over a verified schema

A YOLOv8 detector [9] runs over the full frame. The serving layer is schema-agnostic: if fine-tuned weights are present, their class vocabulary is read from the weights themselves; otherwise the stock COCO model serves as a fallback with COCO hazard semantics. The trained schema (AV-7) names seven classes chosen for navigation consequence: person, chair, table, door, stairs_up, dustbin, and stairs_down.

Hazard status is a semantic role assigned in code to a class name — deliberately not a visual class, since a generic "obstacle" category has no consistent appearance for annotators to agree on. YOLOv8 was selected for its single-stage speed on CPU-class hardware and its mature training tooling; no architectural novelty is claimed for the detector itself.

B. Oriented text reading and symbol resolution

Text detection currently uses the CRAFT detector via EasyOCR [12], which yields rotated quadrilaterals natively; each quadrilateral is perspective-warped to horizontal before recognition by the stock CRNN recognizer [10]. No recognizer fine-tuning is claimed. The pipeline is detector-agnostic: a YOLOv8-OBB text detector can replace CRAFT by placing weights at a designated path, and conversion tooling for ICDAR-2015 [13] and SynthText [15] is implemented and tested; training that detector is future work and is reported here as pipeline design, not as a trained result.

Signs resolve through two routes: a detected class implies its symbol, and recognized text is matched against a symbol vocabulary by whole-word comparison. The whole-word rule exists because substring matching once announced a restaurant *MENU* board as a washroom — "men" is a substring of both MENU and WOMEN — and the regression is pinned by a test. Five pictogram sign classes exist in the annotation vocabulary but hold no labelled boxes yet, so no trained symbol recognition is claimed; text-free pictograms are the stated next annotation target precisely because they are the case OCR-only readers cannot serve.

C. Spatial awareness in walking steps

Direction is derived from frame thirds (left / ahead / right). Distance uses two independent monocular estimators grounded in standard projective geometry [14]: (i) a pinhole model over per-class real-world height priors, and (ii) a ground-plane model that projects the bounding-box foot point through the camera's vertical field of view, corrected by the device pitch reported by the gyroscope. The two estimates are fused by taking the **minimum**: under-estimating distance is the safe error for a person who cannot visually confirm the estimate. The result is spoken in walking steps — the unit the user can act on — rather than metres. Because browser-reported field of view is unreliable across devices, the client includes a one-tap calibration: the user places any known object four steps ahead, and the true vertical field of view is solved from that single measurement and persisted per device.

D. Safety-priority engine and verified capability

The priority engine composes one sentence per frame in a fixed order: **critical alert → hazards → user-requested object → symbols/signs → scene summary** (Fig. 2). The single critical class, stairs_down, produces an interrupting "Warning — stop and proceed carefully" utterance that preempts all other content. Hazard-class objects that are distant are demoted to ordinary objects so that only proximate danger interrupts; the client additionally vibrates on hazards and suppresses its own microphone while speaking, so the recognizer does not transcribe the system's own output.

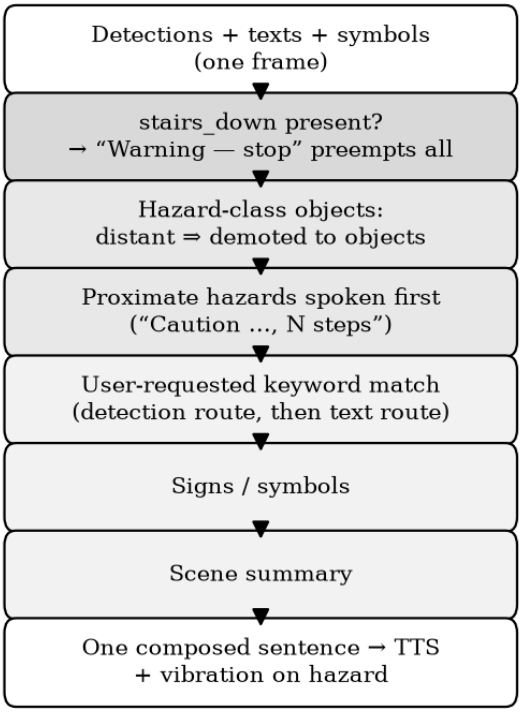

Fig. 2. Priority-scheduled composition of one spoken sentence per frame.

Two verification mechanisms make this pipeline trustworthy across model updates. First, the runtime identifies any weights placed at the custom-model path by their **class names**, never the filename. Weights whose vocabulary matches the navigation schema activate navigation-hazard semantics; COCO weights retain COCO hazard semantics; anything else is refused at startup. The bug this kills is concrete: a COCO model mistaken for the fine-tune would silently stop flagging vehicles, with no error raised to a user who cannot see the difference. Second, the same philosophy is applied *before* training: the split builder refuses a dataset that declares a class with no training boxes, because the resulting model would pass the name check while structurally unable to emit the class it claims. Between the two checks, the health endpoint reports the active schema, whether the critical-alert path is currently servable, and the list of hazard roles the deployed weights cannot raise — capability loss is surfaced as data, not discovered by accident.

V. DATASET AND DATA PREPARATION

The dataset pipeline (Fig. 3) combines self-collected walkthrough footage with public supervision, under two rules: labels move between vocabularies by class *name*, never by index, and no frame-level split is ever taken.

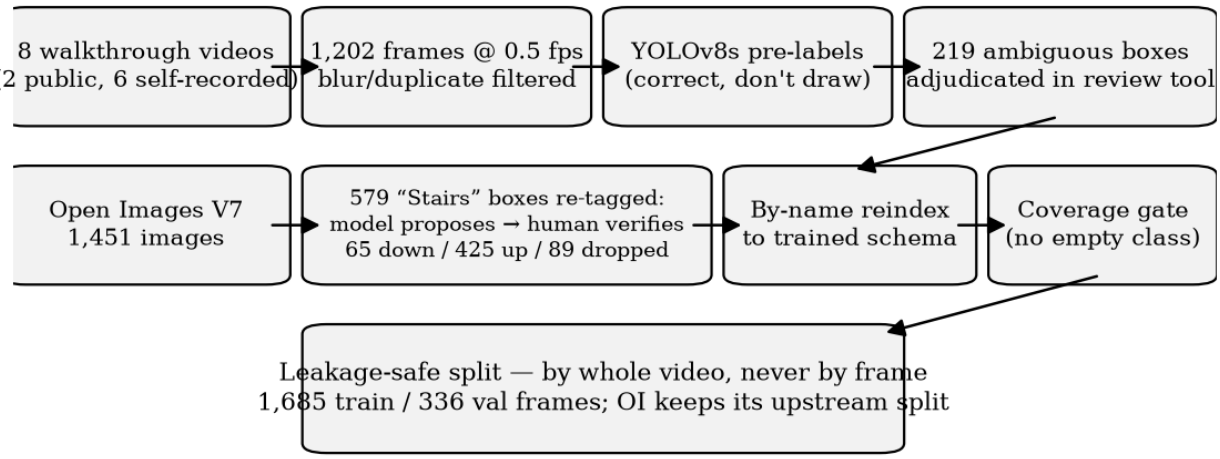

Fig. 3. Dataset preparation pipeline with the human-verified stairs re-tagging protocol.

Corpus. The self-collected corpus comprises 1,202 frames extracted at 0.5 fps from eight indoor walkthrough videos (malls, hospitals, campuses: two public tours, six self-recorded), blur- and duplicate-filtered, and pre-labelled by a stock YOLOv8s so that annotation is correction rather than drawing; 219 ambiguous pre-label boxes were individually adjudicated in a purpose-built review tool. This is supplemented by 1,451 Open Images V7 [11] images supplying volume for door, staircase, and dustbin.

A human-verified critical class. Open Images labels staircases without direction. All 579 imported staircase boxes were re-tagged for this work under a two-stage protocol: a vision-model pass proposed a direction for every box, and a human then reviewed *every box proposed as descending*, confirming 65, demoting 53, and dropping one — the audit record ships with the dataset. The asymmetric protocol reflects asymmetric risk: a wrongly ascending label leaves the status quo, while a wrongly descending label would train the class that triggers the system's only interrupting alert. The 45% correction rate on proposed descending boxes is itself evidence that unreviewed model labels would have been unacceptable for this class. The final corpus carries 65 verified descending-stairs boxes (60 train / 5 validation) — few, but the first supervised signal this class has had, and honestly below the support at which we treat per-class AP as more than directional.

Split. Train/validation splitting is by whole video, never by frame: consecutive 0.5 fps frames are near-duplicates, and a frame-level split leaks validation into training. Open Images contributes its own upstream train/validation assignment, which is honoured rather than re-split. Four self-recorded videos are held out entirely for validation. Table II summarizes the split; seven further names (five pictogram signs, pole, signboard) remain annotation-vocabulary only. Signboard was retired from the trained schema after 50 boxes measured AP@50 of 0.000 — a declared-but-unlearnable class costs macro-mAP while the name-based check still certifies the weights.

TABLE II — Dataset summary (AV-7 split)

	Frames	**person**	**door**	**dustbin**	**stairs_up**	**chair**	**table**	**stairs_down**
Train	1,685	2,533	604	730	399	277	75	60
Validation	336	266	288	30	27	28	17	5

Augmentation during training uses the Ultralytics defaults moderated for a small corpus: mosaic 0.5 (disabled for the final two epochs), rotation ±5°, value jitter 0.4, horizontal flip 0.5. Heavier augmentation was deliberately avoided: a short schedule spends its epochs on distorted frames instead of the real domain.

VI. IMPLEMENTATION

TABLE III — Implementation environment

Component	**Technology**
Server	Python 3.12, FastAPI, PyTorch (CPU), Ultralytics YOLOv8
Text reading	EasyOCR (CRAFT detector + CRNN recognizer)
Client	Plain HTML/JavaScript; Web Speech, vibration, device-orientation, geolocation APIs
Training	YOLOv8n @ 640 px (preliminary, laptop CPU); YOLOv8s @ 832 px (final, Colab T4, in progress)
Packaging	Docker image, verified end-to-end (build, serve, real-frame analysis)
CI	GitHub Actions: full test suite on Ubuntu / Python 3.11
Development host	Intel Core i5-1145G7 laptop CPU (no discrete GPU)

Everything in Table III is exercised by the repository as shipped: the Docker image was built and probed with real frames (health, analysis, oversize-frame rejection), and the continuous-integration run reproduces the full test suite on a platform independent of the development machine.

VII. RESULTS AND DISCUSSION

A. Functional results

The complete pipeline is operational end-to-end with stock weights: live capture, detection, oriented text reading, symbol resolution, step-distance estimation, prioritized speech, voice keyword search, and the calibration flow all function on-device against the deployed server, inside and outside the container. Functional behaviour is pinned by the 152-test suite detailed in §VIII.

B. Experimental results

Model development is staged, and this paper reports the stage each artefact has actually reached. Two **preliminary detectors** are fully measured: AV-6 (YOLOv8n, 640 px, six classes, early-stopped at epoch 36 of 40) and AV-7 (YOLOv8n, 640 px, seven classes including the human-verified stairs_down, trained to the full 40 epochs; best checkpoint at epoch 29). The **final detector** (YOLOv8s, 832 px, 120 epochs, GPU) is training at the time of writing. All evaluation uses the repository's own harness on held-out validation splits, and every fine-tune is paired with a COCO baseline measured on its **own** split: the AV-6 pair shares one split, and the AV-7 pair shares the extended split of Table II.

TABLE IV — Measured results, each pair on its own held-out split (laptop CPU)

Metric	COCO	AV-6 prelim.	COCO†	AV-7† prelim.
Hazard precision (frame)	0.860	0.958	0.936	**0.976**
Hazard recall (frame)	0.252	0.463	0.297	**0.554**
Hazard F1 (frame)	0.389	0.624	0.451	**0.707**
False-alarm frames	6	3	3	**2**
mAP@50	n/a (vocab.)	0.334	n/a (vocab.)	0.365
mAP@50–95	n/a (vocab.)	0.208	n/a (vocab.)	0.250
Detection latency p50 (ms)	119	96	197*	163*
End-to-end latency p50 (ms)	3,576	3,282	4,253*	4,042*

†Extended (AV-7) split. *The AV-7-split pair was measured back-to-back on the evening of the training run and carries residual system load; the two columns of each pair are internally comparable, and the AV-6-split pair was measured on an idle CPU.

The frame-level hazard alert — a frame counts as positive when it holds a proximate hazard-class box, exactly the condition under which the server interrupts the user — is the one row comparable across vocabularies, and it is the row that matters for the person walking: on the same validation split, the AV-7 model raises recall from 0.297 to 0.554 at higher precision, and the AV-6 pair shows the same shape — recall roughly doubles while false-alarm frames fall.

TABLE V — AV-7 per-class results (validation; ⚠ = under 30 boxes, treat as directional)

Class	AP@50	AP@50–95	Deployed P	Deployed R	n_val
dustbin	0.721	0.543	0.381	1.000	30
stairs_down ⚠	0.595	0.515	1.000	0.333	5
door	0.367	0.226	0.548	0.302	288
stairs_up ⚠	0.359	0.138	0.625	0.357	27
chair ⚠	0.299	0.209	0.375	0.250	28
person	0.160	0.092	0.739	0.108	266
table ⚠	0.052	0.026	0.500	0.125	17

Fig. 4 shows the AV-7 training trajectory; Fig. 5 and Fig. 6 show the precision–recall curves and the normalized confusion matrix produced by the evaluation harness.

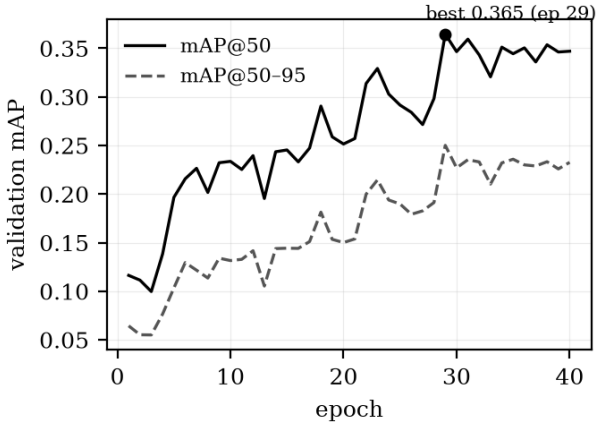

Fig. 4. AV-7 preliminary training: validation mAP versus epoch (YOLOv8n, 640 px, CPU). Best checkpoint at epoch 29.

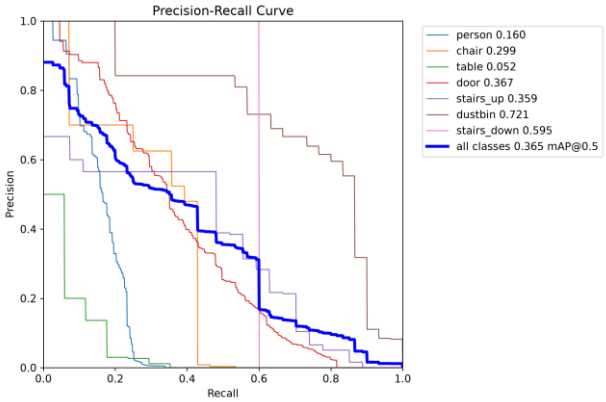

Fig. 5. Precision–recall curves of the AV-7 preliminary detector.

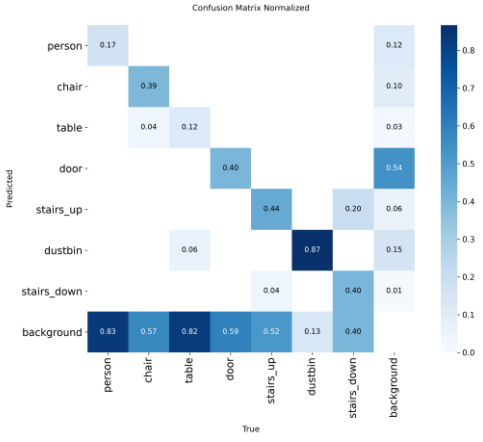

Fig. 6. Normalized confusion matrix of the AV-7 preliminary detector.

The critical class detects. With 60 verified training boxes, stairs_down reaches AP@50 0.595 at precision 1.000 / recall 0.527 (capacity), and at the deployed operating point raises one of its three validation instances with zero false positives. Five validation boxes sit far below any statistically meaningful support — we report the number as a direction, not a result — but the direction is that the system's only interrupting alert path is, for the first time, backed by a detector that has seen its hazard.

A negative result, reported rather than buried. In both preliminary models, person recall at the deployed operating point regresses against the COCO baseline (0.164 → 0.061 for AV-6; 0.108 for AV-7): the fine-tuning corpus holds 2,533 indoor person boxes against COCO's millions. The aggregate hazard metric improves regardless, because newly detected classes compensate — which is exactly why reporting only the aggregate would misrepresent a system materially worse at detecting people. Both preliminary models are therefore **withheld from deployment**. Analysis indicates the deployed per-class confidence threshold (0.62, calibrated for COCO weights) accounts for part of the regression — the AV-7 model's capacity recall for person is 0.135 at maximum-recall settings against 0.108 deployed — and threshold recalibration from the measured precision–recall curves (Fig. 5) is being evaluated alongside the final training run.

Current evaluation status. Pending at the time of writing: final-model (YOLOv8s @ 832) metrics; GPU-tier latency; and the per-class confidence recalibration. All pending values are produced by the repository's own evaluation harness on the splits of §V; none will be sourced from literature.

C. Discussion

The measured pipeline latency (p50 ≈ 3.3 s per frame on an idle laptop CPU) is dominated by text recognition (≈3.1 s); detection contributes under 0.1 s. The system is honestly a paced assistant, not a real-time one, and the latency target for future work is the OCR stage, not the detector. The hazard-first ordering is what turns a 3-second pipeline into a usable assistant: the most consequential information is always the first thing spoken. Dataset limits are equally plain: single-region footage, sparse support for table, chair, and above all stairs_down, and no trained pictogram-sign recognition yet. Distance estimation is monocular and assumes the object rests on the ground plane; the conservative minimum-fusion policy trades accuracy for safety by design. The person-recall regression demonstrates why per-class gates matter more than aggregate metrics in safety-adjacent systems — and why this system's deployment decision is made per class, not per headline number.

VIII. FUNCTIONAL TESTING

All deterministic pipeline logic is pinned by **152 automated tests**, passing locally (Windows, Python 3.12) and in CI (Ubuntu, Python 3.11). Table VI groups them. The suite is regression-oriented: several tests encode bugs that actually occurred — the MENU/WOMEN substring confusion, a portrait-mode distance error, review-tool verdicts silently lost to a single-threaded server — so that none can return unnoticed.

TABLE VI — Test suite summary (152 tests)

Area	Tests	What is pinned
Pipeline logic	40	priority order, critical interrupt, hazard demotion, step distance, deskew, symbol matching
Split construction	21	by-video leakage safety, schema resolution, coverage gating
API limits	18	oversize frames, rate limiting, warmup 503, health capability report
Label remapping	18	COCO→schema by-name mapping, vehicle-box deletion, review queue
Review tooling	19	adjudication merge, crop review server, verdict persistence
Dataset reindex	10	by-name vocabulary migration, background-negative preservation
Stairs re-tag	6	geometry-keyed verdict application, idempotence, decision validation
OBB converters	7	ICDAR-2015 / SynthText / COCO-Text to YOLO-OBB
Training gates	5	coverage refusal before training, schema name check on weights
Evaluation harness	6	metric computation, operating-point accounting
Frame extraction	2	deterministic 0.5 fps extraction and naming

End-to-end behaviour was additionally verified inside the shipped container: health reporting, real-frame analysis with detection, OCR, step distance and keyword search, and rejection of oversize uploads.

IX. LIMITATIONS AND FUTURE WORK

The limitations are stated by measurement, not disclaimer. (i) End-to-end latency is OCR-bound at ≈3 s per frame on CPU; evaluating faster text detection — including the YOLOv8-OBB path whose conversion tooling already ships — and OCR gating are the highest-value optimizations. (ii) The person class regresses at the deployed operating point in both preliminary models; the final GPU training and per-class threshold recalibration target exactly this, and no model ships until person recall is restored. (iii) stairs_down has five validation boxes; its AP is directional until more descending footage is annotated — self-recorded top-of-staircase captures are the only realistic source, since public datasets photograph staircases from below. (iv) Five pictogram sign classes and two further obstacle classes hold no labels yet; annotating them re-activates trained symbol recognition through a one-line schema change, by design. (v) Distance is monocular with a ground-plane assumption; metric depth models are a candidate upgrade. (vi) No user study has been

conducted and none is claimed; a protocol (task completion, collisions, usability scoring [16] against a priority-disabled baseline) is specified for post-training work.

X. CONCLUSION

We presented an assistive vision system that treats *what to say first* and *whether the model can still say it* as first-class engineering problems. The implemented pipeline integrates oriented text reading, navigation-specific obstacle detection, step-metric monocular distance, and prioritized speech, and wraps the model lifecycle in verification: schemas are checked by name at startup, unlearnable classes are refused before training, and the health endpoint reports precisely which alerts the deployed weights can raise. Two measured preliminary detectors more than double frame-level hazard recall over a COCO baseline at higher precision; the second gives the safety-critical descending-stairs class its first detector, trained on labels that survived an asymmetric human-verification protocol that corrected 45% of the machine's proposals for the class that matters most. Both models' person-recall regression — surfaced by the same evaluation discipline — is the documented reason neither is deployed. Final GPU training is in progress; its metrics will complete Tables IV–V without altering the system's architecture or claims.

[[CAMERA-READY: repository URL]]

REFERENCES

[1] World Health Organization, *World Report on Vision*. Geneva: WHO, 2019.

[2] X. Zhou et al., "EAST: An efficient and accurate scene text detector," in *Proc. IEEE CVPR*, 2017, pp. 5551–5560.

[3] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in *Proc. IEEE CVPR*, 2019, pp. 9365–9374.

[4] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," in *Proc. AAAI*, 2020, pp. 11474–11481.

[5] M. He et al., "MOST: A multi-oriented scene text detector with localization refinement," in *Proc. IEEE CVPR*, 2021, pp. 8813–8822.

[6] A. Bhowmick and S. M. Hazarika, "An insight into assistive technology for the visually impaired and blind people: State-of-the-art and future trends," *J. Multimodal User Interfaces*, vol. 11, no. 2, pp. 149–172, 2017.

[7] S. Real and A. Araujo, "Navigation systems for the blind and visually impaired: Past work, challenges, and open problems," *Sensors*, vol. 19, no. 15, p. 3404, 2019.

[8] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014, pp. 740–755.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[10] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, 2017.

[11] A. Kuznetsova et al., "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," *Int. J. Comput. Vis.*, vol. 128, pp. 1956–1981, 2020.

[12] JaidedAI, "EasyOCR: Ready-to-use OCR," 2020. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[13] D. Karatzas et al., "ICDAR 2015 competition on robust reading," in *Proc. ICDAR*, 2015, pp. 1156–1160.

[14] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[15] A. Gupta, A. Vedaldi, and A. Zisserman, "Synthetic data for text localisation in natural images," in *Proc. IEEE CVPR*, 2016, pp. 2315–2324.

[16] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[17] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788.